System Architecture Document

Project: Web-Based Educational Platform for Centralized Academic Resources **Document type:** System Architecture Document **Version:** 1.0

1. Purpose and Scope

This document describes how the platform is structured: its layers, components, data model, security design, deployment and the technology choices behind them. It translates the Project Requirements Document into a buildable technical design.

2. Architectural Goals and Principles

Goal	How the architecture supports it
Simplicity	A single web application with clear layers, suitable for a student team to build and maintain
Organized content	A hierarchical data model: Semester → Subject → Module → Resource
Security	Separate public and admin areas, hashed passwords, validated uploads
Extensibility	Modular design so search, video, forums and AI features can be added without rewriting the core
Performance	Pagination, indexed queries, static file serving and optional caching
Low cost	Open-source stack and free-tier-friendly hosting

3. High-Level Architecture

The system follows a **three-tier client–server architecture**.

1. **Presentation tier (client):** Web browser rendering the student portal and the admin dashboard.

2. **Application tier (server):** Web application containing routing, business logic, authentication and file handling.
3. **Data tier:** Relational database for metadata and a file store for uploaded documents.

Request flow (typical download): Browser → Web server / reverse proxy → Application (routing + controller) → Database (resource metadata) → File storage (file) → Response back to browser.

Request flow (admin upload): Admin browser → Authentication check → Upload controller → File validation → File storage + database record → Confirmation shown to admin.

4. Component Architecture

4.1 Presentation Layer

Component	Responsibility
Public pages	Home, semester list, subject list, module page, resource list, lab section, question papers, announcements, feedback, contact
Admin dashboard	Login, overview, content manager, upload form, feedback inbox, announcement manager, settings
Shared UI	Header, footer, navigation, breadcrumbs, responsive layout, form validation messages
PDF viewer	Embedded browser viewer for in-page reading

4.2 Application Layer

Module	Responsibility
Routing / Controllers	Map URLs to actions for public and admin areas
Content service	Create, read, update and delete semesters, subjects, modules and resources
Upload service	Validate file type and size, rename files safely, store, record metadata

Module	Responsibility
Download service	Serve files, optionally log download counts
Auth service	Admin login, session management, password hashing, access control
Feedback service	Store feedback and contact messages, mark read, delete
Announcement service	Manage and publish announcements
Search service	Basic keyword search over titles and subjects (Phase 1); extensible for advanced search
Notification service (optional)	Email admin on new feedback

4.3 Data Layer

Component	Responsibility
Relational database	Stores structure, resource metadata, admin accounts, feedback and announcements
File storage	Stores uploaded files on server disk or object storage, referenced by path
Backup job	Scheduled backup of database and files

5. Technology Stack

The stack below is a recommendation. The design is technology-neutral, so the team may substitute equivalents.

Layer	Recommended option	Alternatives
Front end	HTML5, CSS3, JavaScript, Bootstrap or Tailwind CSS	React
Back end	Node.js with Express	PHP with Laravel, Python with Django or Flask

Layer	Recommended option	Alternatives
Database	MySQL or PostgreSQL	MongoDB, SQLite for prototyping
File storage	Server disk (Phase 1)	AWS S3, Cloudinary, Firebase Storage
Authentication	Session or JWT with bcrypt-hashed passwords	OAuth for future student login
Web server	Nginx	Apache
Version control	Git and GitHub	GitLab
Hosting	VPS, Render, Railway or similar	Institution server

6. Data Architecture

6.1 Entities and Key Fields

Entity	Key fields
Admin	admin_id, name, email, password_hash, role, created_at
Semester	semester_id, name (e.g., Semester 3), display_order
Subject	subject_id, semester_id (FK), code, name, description
Module	module_id, subject_id (FK), module_number, title
ResourceType	type_id, name (Notes, Question Paper, Lab Manual, Program, Other)
Resource	resource_id, title, module_id (FK, nullable), subject_id (FK), type_id (FK), exam_year (nullable), file_path, file_format, file_size, is_published, download_count, uploaded_by (FK), uploaded_at, updated_at

Entity	Key fields
Announcement	announcement_id, title, body, is_active, created_by (FK), created_at, expires_at
Feedback	feedback_id, name, email, category, message, rating, is_read, created_at
ContactMessage	message_id, name, email, subject, message, is_read, created_at

6.2 Relationships

- One Semester has many Subjects.
- One Subject has many Modules.
- One Module has many Resources.
- Question papers and lab resources attach directly to a Subject, so Resource carries both subject_id and an optional module_id.
- One ResourceType classifies many Resources.
- One Admin uploads many Resources and announcements.

6.3 Indexing and Integrity

- Foreign keys with cascading rules, restricting deletion of a subject that still has resources unless confirmed.
- Indexes on subject_id, module_id, type_id, and a text index on title for search.
- Unique constraint on admin email and on subject code within a semester.

7. Interface Design

7.1 Public Routes (examples)

Route	Purpose
/	Home page with announcements
/semesters	List of semesters
/semesters/:id	Subjects of a semester

Route	Purpose
/subjects/:id	Modules and resource types of a subject
/modules/:id	Resources of a module
/resources/:id/view	In-browser PDF viewer
/resources/:id/download	File download
/lab, /question-papers	Dedicated resource sections
/search?q=	Keyword search
/feedback, /contact	Forms

7.2 Admin Routes (examples, authentication required)

Route	Purpose
/admin/login, /admin/logout	Authentication
/admin/dashboard	Overview
/admin/semesters, /admin/subjects, /admin/modules	Structure management
/admin/resources, /admin/resources/new	Resource list and upload
/admin/feedback	Feedback and messages inbox
/admin/announcements	Announcement management

7.3 API Option

If a React front end is chosen, the same routes are exposed as a REST API (for example GET /api/semesters, POST /api/admin/resources) returning JSON, with admin endpoints protected by token authentication.

8. Security Architecture

Concern	Control
Authentication	Admin-only login; bcrypt or argon2 password hashing; session timeout; login attempt limiting
Authorization	Middleware guarding every /admin route; role checks if multiple admin levels exist
Input validation	Server-side validation and sanitization on all forms and query parameters
Injection attacks	Parameterized queries or ORM; output escaping to prevent XSS
CSRF	Anti-CSRF tokens on all state-changing forms
File upload safety	Allow-list of extensions and MIME types, maximum size limit, random file names, files stored outside executable paths
Transport	HTTPS with TLS certificate in production
Spam protection	CAPTCHA or rate limiting on feedback and contact forms
Secrets	Credentials in environment variables, never in source code
Privacy	Store only name and email for messages; display a short privacy note on forms

9. Deployment Architecture

Environment	Purpose
Development	Local machines with a local database
Staging (optional)	Test deployment for review and user testing
Production	Public site

Production layout: Domain/DNS → Nginx reverse proxy with HTTPS → application process (managed by PM2, systemd or a platform service) → database server and file storage volume.

Operations:

- Automated deployment from the Git main branch (CI/CD through GitHub Actions is recommended).
- Daily database backup and weekly full file backup, with copies kept off the server.
- Application and error logs retained for troubleshooting.
- Basic uptime monitoring.

10. Performance and Scalability Design

- Pagination on all resource lists.
- Browser caching for static assets and compressed responses.
- Optional CDN or object storage for large files as traffic grows.
- Stateless application design so multiple instances can run behind a load balancer if needed.
- Database indexes on frequently filtered columns.
- Upload size limit (for example 25 MB per file) to protect storage and bandwidth.

11. Extensibility Plan (Future Features)

Future feature	Architectural provision
Advanced search	Replace basic search service with full-text search (database full-text or Elasticsearch/Meilisearch); extract PDF text at upload time
Video lectures	Add Video entity linked to Module; store on object storage or embed from a video host
Discussion forums	New Forum, Thread, Post entities; requires student authentication and moderation roles
AI academic assistant	Separate AI service that retrieves from indexed resources and calls an LLM API; exposed through a chat endpoint

Future feature	Architectural provision
Student accounts	Add User entity and role-based access, bookmarks and history

12. Design Decisions and Trade-offs

Decision	Rationale	Trade-off
Monolithic application for Phase 1	Fastest to build and deploy for a small team	Less independent scaling; mitigated by modular code
Relational database	Content is strongly hierarchical	Less flexible schema than NoSQL
Local disk file storage first	Zero extra cost and simple	Must migrate to object storage if storage or traffic grows
No student login in Phase 1	Lowest barrier to access	No personalization until later phase

13. Requirements Mapping

Requirement area	Addressed by
Hierarchical browsing (FR-U2)	Data model in section 6, public routes in 7.1
View and download (FR-U3, U4)	Download service, PDF viewer
Admin content management (FR-A2–A4)	Content and upload services, admin routes
Feedback and announcements (FR-U7, A5, A6)	Feedback and announcement services
Security (NFR-4)	Section 8
Responsiveness (NFR-3)	Responsive presentation layer
Future features	Section 11